IITM Explainable Slotbook: Design Document

CS5013 course project • Nishanth S (EE23B049) and Sparsh Gupta (EE23B117)

We wrote this up in November 2026 from our October design notes, the prompt log (P001–P017) and the commit history. It records the plan we worked from, so it is written in the future tense.

---

1 The problem

Every semester our department has to give each course a lecture slot on the institute timetable grid. In practice someone starts from last year's slotbook and edits it by hand. The reasons behind each choice, like a professor who can't teach on Monday mornings or two core courses that third-years take together, live in people's heads and in old emails. When a slot changes, nobody can say why. When two courses clash, nobody can say what it would cost to move one of them.

We want a tool that builds the slotbook from the rules and preferences directly, and can explain every decision it makes. Three things shape the design. First, it really is a constraint problem: some rules can never be broken (a course must not clash with its own lab), while others are wishes with a cost (an instructor's preferred slots, keeping last year's slot, fewer student clashes). Second, people will only trust a timetable they can understand, even an optimal one. Third, most of the inputs, especially student course choices, have to be collected before anything can be solved.

2 Goals and non-goals

By the end of the course the tool should produce a complete slotbook that never breaks a hard rule and keeps a documented penalty as low as it can. Every assignment should come with an explanation of why the course got that slot and why it didn't get each of the others. Running it twice on the same input should give the same answer. We also want a small web portal where students rank courses and instructors state their preferences, so the inputs don't have to be typed up by hand. Later on, we want to warn students who pick a course without having done its prerequisites.

There are some things we are deliberately not doing. No language model will decide or explain a slot; every reason must come from the constraints themselves. We won't allocate rooms, since each course keeps the room it has. The demo portal will have no login, and we'll say so on every page. Prerequisites will produce warnings, not rejections, because departments do waive them and a self-uploaded grade card proves nothing. All student and instructor data in the repository will be synthetic.

3 Requirements

The engine reads five CSV files: courses, slots, hard conflicts, instructor preferences and student preferences. It validates them before doing anything else. Every problem is reported with a specific code such as `FIXED_CONFLICT` or `UNKNOWN_SLOT`, and invalid input is never solved. Each course then gets exactly one slot from its allowed set, or its fixed slot if it has one, subject to four hard rules: no lecture over the course's own lab, no overlap between protected pairs of courses, no instructor teaching two overlapping meetings, and no two lectures in the same room at the same time. Overlaps are worked out from actual meeting times on the official IITM grid, because slots with different letters can still overlap.

The outputs are `generated_slotbook.csv` and `assignment_explanations.csv`, available from a command-line tool and a small JavaFX viewer. The portal exports data in exactly the engine's CSV format, so the two halves meet at plain files.

Beyond features, we care about a few properties. Explanations must be derivable from the model alone, so they are the same whichever solver ran. Hard rules are checked by an independent checker rather than trusted from the solver. Anything stored by the portal must already satisfy the engine's input rules, and simultaneous submissions must neither fail nor corrupt data. We keep personal data to a minimum and run everything on free tiers.

4 Design

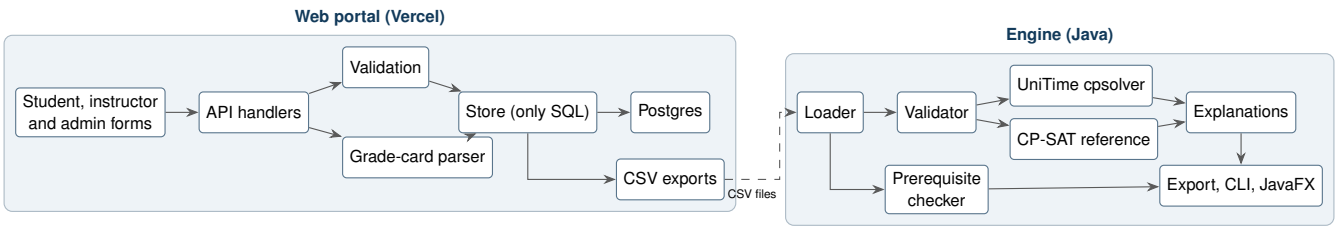


Figure 1: How the pieces fit. The only link between portal and engine is a set of CSV files.

4.1 Engine

Data flows one way: load, validate, solve, explain, export. The Java packages follow the same direction. The `model` package holds immutable records and depends on nothing, while `io`, `validation`, `solver` and `explain` depend only on `model`. The CLI and the viewer share one service class, so neither contains solver logic.

4.2 What we minimise

The penalty lives in a single `Objective` class that every solver and the explanation code share. A course costs the instructor's stated penalty for its slot (0 to 100, and 1 if they said nothing about that slot), plus 2 if it moves away from last year's slot. Each overlapping pair of courses costs 3 per overlapping meeting, multiplied by the pair's weight. That weight adds up, over every student taking both courses, $\max(1,\ 4 - \text{worse rank})$. A clash between two students' top choices therefore costs more than one involving a seventh choice. We kept the numbers small on purpose, so each term can be explained in one sentence.

4.3 Solvers

Both solvers sit behind a `TimetableSolver` interface. We will build an exact CP-SAT model first, using OR-Tools. It can prove optimality on small inputs, which gives us something to measure against. After that we will wrap UniTime's *cpsolver* library, the search engine behind a real university timetabling system, and make it the default because it scales. We'll use the library rather than a UniTime server, and check that its score equals our own objective on every test input. CP-SAT stays in the project as a reference, so the heuristic's gap from the optimum is something we measure rather than assume.

4.4 Explanations

Explanations are computed after solving, from the problem and the final assignment. For the chosen slot we list the rules it satisfies and the penalties it pays. For every other slot we name the hard rule it would break, or give its estimated extra cost if only that course moved. That cost is labelled as an estimate, because it isn't a full re-solve.

4.5 Portal and storage

The portal is a set of static pages with serverless API functions on Vercel. Requests pass through pure validation code into a store module, which is the only place that touches the database, and every query is parameterised. For the first demo we'll store submissions in a Google Sheet, because it's free and needs no setup. Once we need relational rules and several people writing at once, we'll move to Postgres on Neon. There, the same rules that live in our validation code are repeated as foreign keys and `CHECK` constraints. Replacing a submission happens in one transaction, and schema changes are numbered migrations.

4.6 Prerequisites

Instructors list the prerequisites for their course, and students can upload the PDF grade card from the institute portal. We read course, grade and term from it and throw the file away. U, W and I don't count as completed, and a retaken course counts if any attempt passed. A missing prerequisite shows up as a warning when the student submits, on the admin page and in the CLI. It never blocks anything, and the solver never sees it. The portal (JavaScript) and the CLI (Java) implement the same rule, so both test suites check one shared file of expected warnings. Grade cards are only a first source of completions. Verified records from the academic section can be added later without changing the forms or the schema.

5 Choices we considered

Question	Options	What we picked and why
How to solve	Greedy heuristic, MIP, CP-SAT, UniTime server, cpsolver library	CP-SAT first for provable answers, then cpsolver for scale. A home-made heuristic would be neither provable nor proven.
How to explain	Language model, solver logs, recompute from the model	Recompute. It's deterministic and checkable, and works for both solvers. A language model can sound right and be wrong.
Clash detection	Same slot letter, real meeting times	Real times, since some lab and lecture slots overlap without sharing a letter.
Storage	Files on Vercel, Google Sheets, Neon Postgres, Supabase, SQLite	Sheets for the demo, then Neon. Vercel's disk doesn't persist, and Supabase brings features we don't need.
Prerequisites	Enforce, warn, ignore	Warn, because records are self-reported and waivers happen.

6 How we will test it

We test against oracles we write separately, not by asking the code whether it is right. Unit tests cover the parsers, every validation code, the objective and the explanations. For solver correctness, an independent constraint checker and penalty calculator check both solvers on fixed and randomly generated scenarios, and on small cases an exhaustive brute force confirms that CP-SAT really is optimal. Every scenario is solved by both solvers, and the gap is recorded. The portal gets contract tests against the engine's CSV headers, plus integration tests on a real Postgres for rollback and concurrency. Finally, stress tests throw seeded random input, corrupted PDFs and hundreds of simultaneous requests at it. These must never produce a server error or a half-written submission.

We never assert one exact timetable when several are equally good. Every random test is seeded so a failure can be replayed, and every bug fix gets a test that we first watch fail on the old code. GitHub Actions runs everything on each push and pull request.

7 How we will work

Most of the code will be written with an AI assistant, so we've set ourselves rules. Every piece of work starts with an entry in `prompts/PROMPT_LOG.md`, written before any code. It records the request as given, a structured restatement (CoSTAR), and a list of things the output must *not* do, noting whether each came from us, from planning or from review. AI output counts as untrusted until the tests and checks pass, and we record what we rejected. Commits are small and pass the tests, larger changes go through a pull request, bugs are written up as we find them, and documentation is versioned alongside the code.

8 Plan and risks

We plan seven milestones:

M1 The v1 engine with CP-SAT, explanations, CLI and viewer.
M2 A testing framework with oracles and generated scenarios.
M3 v2, with cpsolver as the default and real IITM meeting times.
M4 The demo portal on Google Sheets.
M5 The mid-evaluation.
M6 v3, with Postgres, prerequisites and a man page.
M7 Stress testing and deployment.

The risks we take most seriously:

- **The heuristic falls short of the optimum.** CP-SAT stays as a yardstick.
- **Our objective drifts away from cpsolver's score.** A test checks that the two scores are equal.
- **Real student data ends up in the repository.** Test grade cards are generated and marked, and a check rejects any that aren't.
- **Simultaneous submissions corrupt the data.** Transactions and database constraints guard against it, and the stress tests check it.

The missing login is a known gap. We won't collect real data until there is institute sign-in, consent text and a retention policy.

Some questions are still open. Who would supply verified course records, and in what format? Should some prerequisites ever block a submission, with an override? Is our rank weighting right, or should lower choices count for even less? And should room allocation eventually become part of the model?